LandingAI

Deeplearning.AI – The AI Developer Conference 2026

Agentic Document Extraction for Agentic Loan Origination

Hierarchical Multi-Agent Document Intelligence using Google ADK + Claude

LandingAI ADE | Google ADK | Claude

Agenda

01 Who is LandingAI?

02 Agentic Loan Origination + ADE Demo

03 Solution Overview

04 Hierarchical Agent Architecture

05 System Architecture + Agent Orchestration

06 Agent Logic & Multi-Agent Best Practices

07 Case Studies

08 Q&A

COMPANY OVERVIEW

The Agentic Document Platform for Enterprises

LandingAI provides a platform for building and deploying document AI workflows that extract structured data from complex enterprise documents.

Andrew Ng	1B+	1,200+	50+	99.16%
Founded By	**Images and docs processed**	**Customers**	**Fortune 500 Customers**	**Accuracy**
Co-founder of Coursera, Google Brain				#1 DocVQA benchmark

Offering

AGENTIC DOCUMENT EXTRACTION (ADE)

Parse, split, and extract from complex documents via API. Powers KYC, claims, lending, underwriting, and compliance workflows across regulated industries.

- **APIs**
  Parse · Extract · Schema Building · Split
- **Developer-First**
  REST APIs and Python or TypeScript libraries for rapid integration
- **Any Document Type**
  No templates—schema-driven extraction
- **Production Ready**
  SOC 2 · HIPAA · Enterprise SLAs

Differentiators

Vision-First AI
Analyzes full visual context vs. premature text extraction. Every output links to exact source coordinates for auditability.

Agentic Architecture
Multi-step reasoning chains for complex document workflows. Planning, deciding, and verifying until quality thresholds are met

Proprietary DPT-X Models
Document Pre-trained Transformer family delivers 99.16% accuracy.

Deployment Flexibility
SaaS, VPC, and On-Premise deployment for data-sensitive industries.

Agentic Loan Origination + ADE Demo

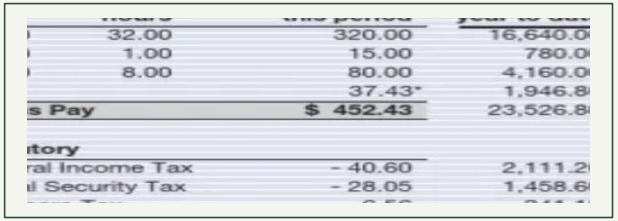
LandingAI

Visual Grounding – Seeing Exactly What the Agents See

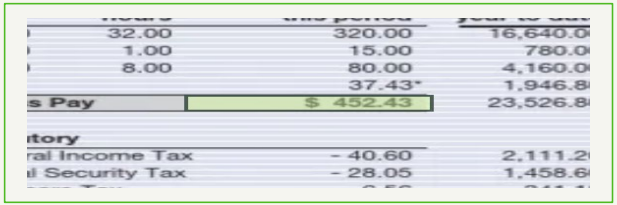
LandingAI's ADE returns bounding box references for every extracted field – fully autonomous & auditable

Pay Stub

`gross_pay` → $452.43



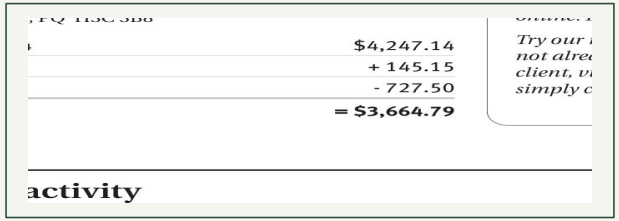
↓ *ADE field extraction + grounding*



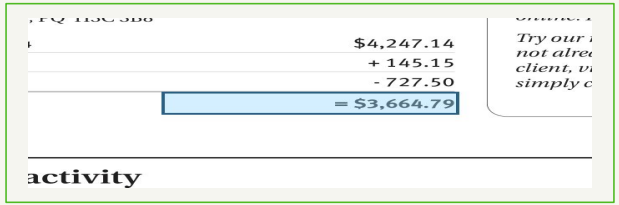
`conf` 85% chunk ref: 0-u · page 1

Bank Statement

`balance` → $3,664.79



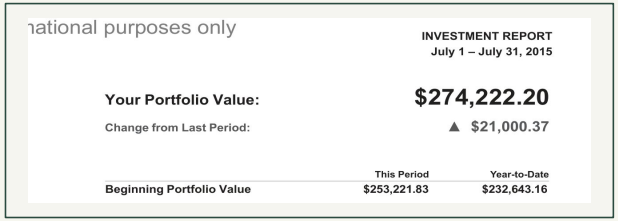
↓ *ADE field extraction + grounding*



`conf` 85% chunk ref: 1-e · page 2

Investment Statement

`total_investment` → $274,222.20



↓ *ADE field extraction + grounding*

`conf` 85% chunk ref: 3c7f6001 · page 3

Each highlighted region is a bounding box from `parse_result.grounding[chunk_id]` — enabling pixel-level audit trails for every loan decision.

LandingAI

Solution Overview

What We Built & Why

PROBLEM

Loan officers manually review mixed-type document packets — pay stubs, bank statements, investment statements — to verify income, assets, and DTI. Error-prone, slow, not auditable.

SOLUTION

Five-agent system: classify, parse, extract, decide — a Manager Agent reviews the decision for consistency and overrides when needed — and a Chat Agent powered by Claude Haiku lets users explore scenarios in plain English.

TECH STACK

landingai-ade	google.adk	anthropic	FastAPI	Pydantic
parse + extract	Agent orchestration	chat (Claude Haiku)	REST + async	typed schemas

5 Hierarchical agents

3 Document types

43% DTI threshold (QM)

6 Fraud & validation checks

4 Risk levels (LOW → CRITICAL)

0 Re-uploads to re-analyze & interact

Hierarchical Agent Architecture

Manager Agent orchestrates & reviews – subagents execute specialized tasks for governance and auditability

Manager Agent

Orchestrator · Reviewer · Risk Classifier · Escalation Gate

dispatch(ctx)

Agent 1

Parse & Split

```
parse_and_split()
client.parse(doc, split='page')
client.extract(DocType schema)
group_pages_by_doc_type()
```

Agent 2

Field Extraction

```
extract_fields()
pydantic_to_json_schema()
client.extract(schema,
markdown)
caches in docs[doc_id]
```

Agent 3

Loan Decision

```
decide_loan()
DTI ratio check > 43%
Income / fraud sanity
Reserves ≥ 10% of loan
```

Chat Agent

Plain-English Q&A

```
POST /chat (msg, ctx)
Claude Haiku via Anthropic API
grounded in docs + jobs
explains decisions to user
```

Cache

Session Store

```
docs[doc_id]
extraction results
jobs[job_id]
decision + review
```

↑ *review_result + risk_level + override*

FINAL OUTCOMES APPROVED | DENIED | OVERRIDE | ESCALATE

RISK LEVELS → LOW | MEDIUM | HIGH | CRITICAL

LandingAI

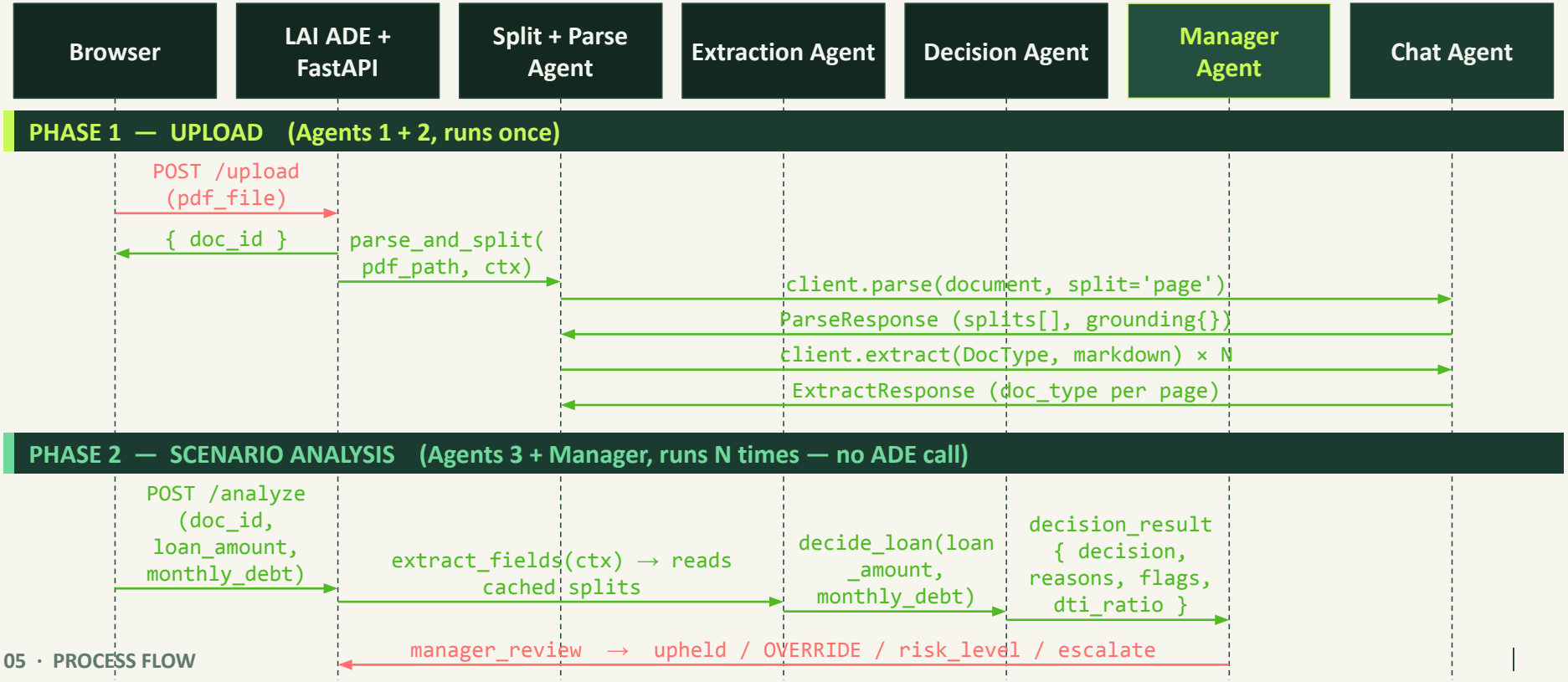
System Architecture

Component topology across all layers

Browser	**Upload** Extract Fields	**Progress** Polling	**Scenario** Run Analysis	**Results** History × N	**Chat** Plain-English Q&A
API Layer	`POST /upload` `main.py`	`GET /upload-status/{id}` `asyncio status`	`POST /analyze` `creates jobs[job_id]`	`GET /result/{id}` `fetch decision`	`POST /chat` `→ Claude Haiku`
Agent Layer	**Agent 1** Parse & Split *agents.py*	**Agent 2** Extraction *agents.py*	**Agent 3** Decision *agents.py*	**Manager Agent** Review + Cache *agents.py*	**Chat Agent** Claude Haiku *chat.py*
SDK / Utils	`LandingAIADE` `.parse() · .extract()`	`anthropic.Client` `Claude Haiku`	`pydantic_to_json_schema()` `typed JSON schemas`	`ParseResponse / ExtractResponse` `response models`	`ade_utils.py` `group_pages_by_doc_type()`

Agent Orchestration – Process Flow

Sequence Diagram – upload phase, analysis phase with manager review, and chat advisor



LandingAI

Agent Logic Detail

Implementation internals for all four agents

Agent 1
Split & Parse

- client.parse(document=f, split='page')
- ParseResponse → .splits[] + .grounding[]
- Per-page: extract(DocType schema, markdown)
- extraction['type'] → doc type string
- group_pages_by_document_type() merges runs
- Writes parse_result + split_docs to ctx

```
with open(pdf, 'rb') as f:
  pr = client.parse(
    document=f, split='page')

for split in pr.splits:
  cls = client.extract(
    schema=doc_type_json,
    markdown=split.markdown)
```

Agent 2
Extraction

- Iterates split_documents{} from Agent 1
- SCHEMA_PER_DOC_TYPE lookup per doc_type
- pydantic_to_json_schema() → JSON str
- client.extract(schema, combined_markdown)
- Stores extraction + extraction_metadata
- Cached in docs[doc_id] → free to re-use

```
for name, doc in \
    split_docs.items():
  schema = pydantic_to_json_schema(
    SCHEMA_PER_DOC_TYPE[doc_type])
  res = client.extract(
    schema=schema,
    markdown=doc['markdown'])
```

Agent 3
Loan Decision

- Reads extractions from ctx (no API call)
- DTI = (debt + est_pmt) / monthly_income
- Deny: DTI > 43% | neg balance | low reserves
- Flag: net_pay > gross_pay (fraud indicator)
- Flag: investment_year in future
- Returns decision, reasons, flags, dti_ratio

```
monthly_income = gross_pay * 26/12
r, n = 0.07/12, 360
pmt = loan*r*(1+r)**n/((1+r)**n-1)
dti = (debt + pmt)/monthly_income

if dti > 0.43: deny = True
if net_pay > gross_pay: flags++
```

Manager
Review

- Receives Agent 3 output → re-evaluates
- Risk scoring: DTI + fraud_flags → 0-10 score
- Override: fraud flag + APPROVED → DENIED
- Override: zero fields + APPROVED → DENIED
- Borderline DTI (38-43%) → escalate flag
- Returns upheld, override, risk, escalate

```
if flags and \
   decision == 'APPROVED':
  override = 'DENIED'
  escalate = True

risk = score_risk(
  dti_ratio, fraud_flags)
# LOW/MEDIUM/HIGH/CRITICAL
```

Manager Agent – Deep Dive

Override logic, risk scoring, and escalation rules

OVERRIDE DECISION TREE

Override #1 `fraud_flags > 0 AND Agent 3 = APPROVED`

→ *force DENIED + escalate*

Override #2 `pay_stub missing AND bank_statement missing`

→ *force DENIED + escalate (no evidence to underwrite)*

Override #3 `zero extracted fields AND Agent 3 = APPROVED`

→ *force DENIED (extraction failure)*

Escalate `DTI in 38-43% AND no fraud flags AND reserves present`

→ *uphold decision but flag for human review*

Uphold `no override conditions triggered`

→ *decision stands, set risk_level, return*

RISK SCORE COMPOSITION (0–10)

DTI > 50%		+4
DTI 43–50%		+2
DTI 36–43%		+1
2+ fraud flags	*escalate*	+4
1 fraud flag		+2

- score ≥ 7 → CRITICAL
- score ≥ 4 → HIGH
- score ≥ 2 → MEDIUM
- score < 2 → LOW

MANAGER_REVIEW OUTPUT SCHEMA

```
{
  upheld: bool,
  override_decision: 'APPROVED' | 'DENIED' | None,
  review_notes: List[str],
  risk_level: 'LOW' | 'MEDIUM' | 'HIGH' | 'CRITICAL',
  escalate: bool
}
```

Fraud & Validation Checks

Six deterministic rules enforced in Loan Decision Agent (3) – no AI calls, every outcome is auditable

01

FRAUD + DENY

agents.py · lines 463–468

Net Pay > Gross Pay

Classic tamper signal — net cannot exceed gross on a legitimate pay stub.

```
if net_pay > gross_pay:
    flags.append('fraud')
    deny = True
```

02

DENY

agents.py · lines 457–459

Zero or Missing Gross Pay

Income of $0 or None means we have nothing to underwrite against — hard stop.

```
if gross_pay is None or
   gross_pay <= 0:
    deny = True
```

03

FLAG

agents.py · lines 522–526

Future Investment Year

Investment statement year is after today — likely fabricated or typo'd.

```
if investment_year >
   datetime.now().year:
    flags.append('future_year')
```

04

DENY

agents.py · lines 497–499

Negative Bank Balance

Negative balance implies overdraft — insufficient liquid reserves to underwrite.

```
if balance < 0:
    flags.append('neg_balance')
    deny = True
```

05

DENY

agents.py · lines 443–448

Missing Required Documents

No pay stub or bank statement means zero evidence — cannot underwrite at all.

```
if not pay_stub or \
   not bank_statement:
    deny = True
```

06

DENY

agents.py · lines 508–515

Insufficient Reserves

Borrower's investments must cover at least 10% of the loan amount.

```
if total_investment <
   loan_amount * 0.10:
    deny = True
```

All six checks run in pure Python on cached extractions — **zero ADE / LLM calls**. Triggering any DENY rule short-circuits Agent 3; FLAGS feed the Manager Agent's risk score.

Best Practices for Multi-Agent Systems

Hardening the pipeline against hallucinations, prompt injection & silent failures

Level 1
Agent Level
Intra-agent safety

Prompt Injection Guard
_sanitise_markdown() strips injection patterns from PDF text before each extract() call

Context Window Cap
Markdown truncated at 12,000 chars — prevents oversized API payloads silently failing

Field-Range Validation
Extracted numerics checked against plausible bounds (e.g. gross_pay 0–500k)

Dynamic Year Bound
datetime.now().year replaces hardcoded 2026 — anomaly checks stay correct over time

Level 2
Agent-to-Agent
Typed inter-agent contracts

Typed Handoff Models
ParseHandoff & ExtractionHandoff validate each message between agents via Pydantic

Validation on Receipt
Every agent asserts its expected input is present — pipeline halts with a clear error if not

Empty-Extraction Guard
ExtractionHandoff raises if ALL documents return empty — catches silent ADE API failures

Confidence Passthrough
ExtractionRecord.confidence carries averaged extraction confidence to the final output

Level 3
Orchestration
Retry, fallback, termination

Retry + Exponential Backoff
_api_call_with_retry() wraps parse() & extract() — 3 attempts: 1.5s → 3s → 6s delays

Structured Final Error
All retries exhausted → raises RuntimeError with attempt count and last exception

Explicit Termination
Unrecoverable errors raise immediately — no silent fallbacks producing unevidenced decisions

Structured Logging
logging.warning() on every retry, redaction, and truncation — observable in production

Level 4
Context / Data Layer
Retrieval quality & grounding

Markdown Length Guard
MAX_MARKDOWN_CHARS = 12,000 per document before extraction call is made

Post-Extraction Validation
FIELD_RANGES dict maps each field to a plausible (lo, hi) bound — out-of-range = flag

Extraction Confidence
_extract_confidence() averages per-field scores from extraction_metadata per document

Warning Propagation
Agent 2 range warnings → job_context → Agent 3 flags → Manager risk score

Each level is independently testable — strip out any layer and the pipeline degrades gracefully with clear error messages.

LandingAI

Customer Case Studies

Automated Client Due Diligence (CDD)

Customer: Tier-1 Global Bank (>$1.5T Assets)
Region: UK / US / EMEA

60%

Upto 60% reduction in manual review time

100+

Hundreds of ***analyst hours*** saved **per week**

$M+

saving per year from hundreds of analyst globally who process documents

AI-driven document intelligence to augment analysts and streamline complex KYC workflows.

"With hundreds of analysts reviewing complex, non-standard customer documents, the cost and inefficiency of manual processing were becoming unsustainable. LandingAI provides a scalable, high-accuracy solution that integrates seamlessly with our existing systems and significantly improves both speed and cost effectiveness."

Read a Case Study

Head of Data & Analytics, **Fortune 100 Financial Services Firm**

LandingAI

© LandingAI All rights reserved.

Doubles Productivity with Global Reimbursement Dossier Automation

Customer: A Fortune 500 Pharma Company

2x

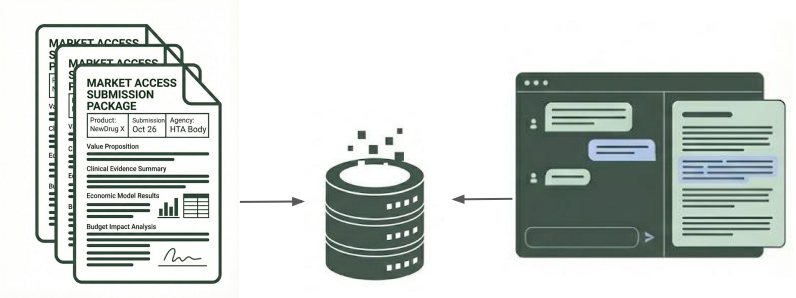
Productivity from the same team and deploying them to higher-value work

50%

Up to 50% faster in auditing effort

$M+

Est. annual impact reducing effort to review and inject content



Evidence Retrieval with Context

"In an increasingly competitive market, we're under pressure to bring therapies to patients faster without increasing headcount. We have to deliver **twice the output at twice the speed.** LandingAI enables us to automate complex **submission review with high accuracy,** allowing our teams to focus on strategic decisions rather than manual document handling."

Operations head, **A Global Life Sciences Leader**

LandingAI

© LandingAI All rights reserved.

Thank you

RESOURCES

- Try your documents in *Playground*
  https://va.landing.ai/demo/doc-extraction
- Demo assets from today:
  ade-helper-scripts/events (AI Dev Day assets)

CONTACT

Sales@Landing.ai